

MobiCloud — Oral Defense Q&A

Simple questions the jury is likely to ask. Focus on **"Why did you choose..."**, broad understanding, and **"What if..."** scenarios.

1 — PROJECT & MOTIVATION

Q1. In one sentence, what problem does MobiCloud solve? General

MobiCloud lets Android phones store and share files with each other directly — without uploading anything to Google Drive, Dropbox, or any cloud service. Your data stays on devices you trust.

Q2. Why did you choose a P2P approach rather than a simple client-server model? Why

A central server is a single point of failure and a privacy risk — if it goes down, nothing works; if it is compromised, all files are exposed. P2P removes that single point: storage is spread across participants, and no central party ever sees the plaintext content.

Q3. Why target Android specifically, and not a desktop or web application?

Why

Mobile phones are the most personal and always-connected devices people own. They accumulate photos, documents, and sensitive data. Building on Android also lets us use the hardware-backed Android Keystore (TEE) to keep cryptographic keys secure — something a web app cannot guarantee.

Q4. What are the main limitations of your current prototype? General

The relay server is still a centralised component (though only for signalling, not data). The cluster size in the demo is set to 2 nodes on the relay side for simplicity. Battery impact

under heavy usage has not been measured on real devices, only simulated. NAT hole-punching is not implemented — peers behind strict NAT must always use the relay.

2 — ARCHITECTURE & DESIGN CHOICES

Q5. Why did you introduce a Super-Peer role instead of a fully flat P2P network? Why

A fully flat network requires every node to know every other node, which scales poorly. The Super-Peer acts as a lightweight coordinator for its cluster: it manages the member list, triggers repairs, and relays election messages. Ordinary peers only need to know their Super-Peer, which reduces overhead significantly.

Q6. Why did you choose the Bully algorithm for Super-Peer election? Why

Bully is simple to implement and converges quickly in small networks (3–10 nodes). It elects the node with the highest reliability score, which naturally promotes stable, well-connected devices. For our cluster sizes, its $O(n^2)$ message complexity is perfectly acceptable. More complex algorithms (Raft, Paxos) would be overkill for a mobile P2P demo.

Q7. Why did you add a relay server if the goal is decentralisation? Why

Mobile phones sit behind NAT (carrier-grade NAT for 4G, home router NAT for WiFi). Two phones cannot reach each other directly without a rendezvous point. Our relay is a *zero-knowledge* relay: it only forwards encrypted bytes and never sees the content. It is a pragmatic trade-off — full decentralisation is theoretically possible with DHT-based signalling, but far more complex to build reliably on mobile.

Q8. Why use Clean Architecture (domain / data / presentation layers) for an Android app? Why

Clean Architecture keeps the core business logic (erasure coding, gossip, election) completely independent of Android. This means the domain layer can be unit-tested without an

emulator. It also makes it easy to swap implementations — for example, replacing the WebSocket transport with a direct TCP transport — without touching the use cases.

Q9. You used Kotlin Coroutines and Flow throughout the app. Why not threads or callbacks? Why

Coroutines give structured concurrency: if a parent job is cancelled, all child coroutines are automatically cancelled. This prevents the common Android bug of background threads leaking after the screen is closed. Flow gives reactive, backpressure-aware streams for peer list updates and network events — much cleaner than callbacks in deeply nested P2P code.

3 — STORAGE & REDUNDANCY

Q10. Why did you choose Erasure Coding (Reed-Solomon) instead of simple replication? Why

Simple replication stores 3 full copies → $3\times$ storage overhead. Reed-Solomon with $k=3$ data + $n=2$ parity stores 5 fragments but each is only $1/3$ of the file size → $5/3 \approx 1.67\times$ overhead. You get the same fault tolerance (can lose any 2 fragments and still recover) at roughly half the storage cost. On mobile, storage is precious.

Q11. What happens if a peer that holds a fragment goes offline permanently?

General

The Super-Peer runs a repair scan every 10 seconds. When it detects a peer is inactive, it checks the DHT to find which blocks that peer was hosting. If a block no longer has enough live copies, the Super-Peer sends a signed "replicate" directive to another live peer that holds a copy, instructing it to send the block to a new destination peer.

Q12. Why did you use a local Room (SQLite) database instead of keeping everything in memory? Why

Mobile apps are killed by the OS at any time. If the DHT and catalog were only in RAM, every restart would mean rediscovering the entire network state from scratch. SQLite

persists the peer list, block locations, and catalog entries across app restarts, so the node can resume quickly without a full gossip re-sync.

4 — SECURITY & PRIVACY

Q13. Why did you choose AES-256-GCM for fragment encryption? Why

AES-256-GCM is an authenticated encryption scheme: it both encrypts the data and produces a 128-bit authentication tag. This means if a stored fragment is tampered with or corrupted, decryption fails immediately — we detect integrity violations without a separate hash step. It is also hardware-accelerated on modern Android devices (AES-NI equivalent on ARM), so the performance cost is minimal.

Q14. Why store keys in the Android Keystore (TEE) instead of in a regular file or database? Why

If the private key is in a file, any app with root access or a backup extraction can steal it. The Android Keystore stores keys inside the Trusted Execution Environment (TEE) — a secure enclave that the rest of the OS cannot read. The private key never leaves the TEE; only the operation (sign/verify) is performed there. Even if the phone is stolen and the storage cloned, the private key cannot be extracted.

Q15. What is the Hashcash mechanism and why did you add it? General

Hashcash is a proof-of-work: before a node is allowed to broadcast itself to the network, it must compute a SHA-256 hash that starts with 18 zero bits. This takes about one second of CPU time. It makes it expensive to create thousands of fake identities (a Sybil attack) — an attacker would need one second of compute per fake node. Legitimate nodes pay the cost once and cache the result.

Q16. How does MobiCloud ensure that the relay server cannot read the stored files? General

Every fragment is encrypted on the sender's device before it is sent anywhere. The relay only ever sees opaque encrypted bytes; it does not have the keys. The encryption keys are derived per-fragment from a master key, and the master key itself is wrapped with the owner's EC public key (ECIES). Even if the relay server is hacked, the attacker gets useless ciphertext.

5 — GOSSIP & SYNCHRONISATION

Q17. What is the Gossip protocol and why did you use it to synchronise the DHT? General

Gossip (epidemic protocol) works like rumour spreading: each node periodically tells a few random neighbours what it knows, and they share back what the first node is missing. There is no master node that maintains the truth. It is decentralised, self-healing, and scales logarithmically — in a 100-node network, a new piece of information reaches everyone in about 7 gossip rounds. It tolerates node failures naturally.

Q18. Why did you use a Bloom Filter inside the Gossip protocol instead of sending the full DHT? Why

A Bloom filter is a compact probabilistic data structure. Instead of listing all 1000 block IDs (possibly several kilobytes), a node sends a 1024-bit (128-byte) filter that summarises which blocks it knows about. The receiver uses it to identify which entries are probably missing on the sender, then requests only those — a delta sync. This drastically reduces bandwidth per gossip cycle.

Q19. Can a Bloom Filter give wrong answers? How does your system handle that? General

Yes — a Bloom filter can produce false positives (saying "I have it" when it does not). It never produces false negatives. In our system, a false positive means a delta request might not ask for a block that is actually missing. The consequence is a slightly delayed sync, not data loss. The next gossip cycle will correct it. We set 3 hash functions on a 1024-bit array, giving a false-positive rate well below 1% for our typical DHT sizes.

6 — SIMULATION & VALIDATION

Q20. Why did you use PeerSim for simulation instead of testing on real devices?

Why

Testing a 100-node distributed system on real phones would require 100 physical devices running simultaneously — impractical for a student project. PeerSim lets us simulate thousands of nodes in a single JVM process, inject churn (nodes joining and leaving), measure convergence time, and validate the election algorithm — all reproducibly and in seconds. Real device tests were done for connectivity validation (3–4 devices).

Q21. What did your simulation tell you about the system's behaviour under churn?

General

At 30% churn (30% of nodes leaving every round), gossip convergence slows but does not break — all nodes eventually agree on the DHT state. The Bully election also remained stable: after a Super-Peer crashed, a new one was elected within 2–3 gossip cycles (about 4–6 seconds with our parameters). Above 50% churn, the network becomes too fragmented for reliable convergence.

Q22. How did you test the security of the system?

General

We reviewed each message type for: (1) authentication — every critical message is signed with EC P-256 and verified on reception; (2) replay attacks — timestamps with a ± 30 -second window; (3) unknown senders — Gossip and election messages from unrecognised peers are silently dropped; (4) DHT poisoning — only known active peers can inject DHT entries via gossip. We did not run a formal penetration test, but these controls address the main attack vectors for a P2P system.

7 — OPEN QUESTIONS & FUTURE WORK

Q23. If you had 6 more months, what would you improve first?

General

Three things: (1) Replace the centralised relay with a fully distributed DHT-based signalling layer so there is truly no central server. (2) Add file versioning and conflict resolution for

concurrent writes. (3) Measure real battery consumption on physical devices and optimise the gossip interval accordingly.

Q24. How does MobiCloud compare to existing systems like IPFS or Bittorrent?

General

IPFS is a full content-addressed filesystem — powerful but very heavy for a mobile app (hundreds of megabytes of dependencies, constant background activity). Bittorrent requires a tracker or DHT bootstrap and is designed for public file sharing, not private encrypted storage. MobiCloud is lighter, mobile-first, fully encrypted end-to-end, and designed for private groups (your own devices or trusted friends), not anonymous public swarms.

Q25. Is MobiCloud ready for production use?

General

No — this is a research prototype. The relay server runs on a free Render instance (which sleeps after inactivity). The cluster size is limited to 2 on the relay demo. There is no user account system, no file browser UI, and no protection against a malicious Super-Peer within a cluster. These are engineering challenges for a production version, not fundamental research obstacles.

8 — WHAT IF...

Q26. What if the relay server goes down completely?

What If

Peers that are already on the same local Wi-Fi network can still communicate directly via mDNS discovery and TCP. Peers on 4G or different networks lose the ability to find each other until the relay comes back — they cannot establish new connections. Files already distributed stay safe on the devices that hold fragments; nothing is lost, just temporarily unreachable across NAT boundaries.

Q27. What if the Super-Peer of a cluster crashes suddenly?

What If

The remaining peers detect the absence of a Super-Peer after about 120 seconds (4 missed heartbeats). If no other Super-Peer is found within 20 more seconds, the Bully election protocol starts automatically. The node with the highest reliability score broadcasts an ELECTION message, waits 3 seconds for any stronger competitor to respond, and if none does, declares itself the new Super-Peer by broadcasting a COORDINATOR message. Total recovery time: under 2 minutes in the worst case.

Q28. What if more than $n=2$ peers holding fragments go offline at the same time? What If

With Reed-Solomon($k=3$, $n=2$), you need at least $k=3$ fragments to reconstruct the file. If 3 or more peers go offline simultaneously and each held a different fragment, the file becomes unrecoverable from that cluster. This is why the repair mechanism is proactive — it runs every 10 seconds and re-replicates fragments the moment a peer goes inactive, before the situation gets worse. The circuit breaker also pauses repairs during network storms to avoid making things worse.

Q29. What if two nodes both think they are the Super-Peer at the same time (split-brain)? What If

This is called a split-brain scenario. We handle it in two ways: first, an existing Super-Peer that receives an ELECTION message immediately replies with a COORDINATOR message to tell the challenger to stand down. Second, the COORDINATOR message itself carries the cluster ID — if two nodes both become Super-Peers with different cluster IDs, new peers joining will pick only one and ignore the other. It is still an edge case we would want to harden further in a production system.

Q30. What if a malicious node joins the network and sends fake DHT entries?

What If

Every node that receives a DHT update via gossip first checks that the sender is a known active peer. Unknown senders are silently rejected. Additionally, every block transfer is acknowledged with a signed ACK that includes the SHA-256 hash of the received ciphertext — if a node injects a fake block ID pointing to garbage data, the download will fail hash

verification. So a rogue node can disrupt availability but cannot silently corrupt data that gets decrypted.

Q31. What if a user's phone is stolen? Can the attacker read their files? What If

No — for two reasons. First, the files are stored as fragments distributed across other devices, not whole on the stolen phone. Second, even the fragments stored on the stolen device are AES-256-GCM encrypted, and the master key is wrapped with the user's EC private key, which lives in the Android Keystore TEE hardware. An attacker cannot extract the private key from the TEE even with physical access. They would need to break AES-256, which is computationally infeasible.

Q32. What if the network has very high churn — many devices joining and leaving constantly? What If

The circuit breaker activates when more than 30% of cluster peers become inactive within a 2-minute window. When open, it queues repair directives in a local buffer instead of sending them immediately, preventing a storm of repair messages that would overload already struggling nodes. Once churn drops below 10%, the circuit breaker closes and queued repairs are flushed. This prevents a cascade failure where repair traffic makes the instability worse.

Q33. What if two users upload the same file at the same time? What If

Each upload produces a catalog entry keyed by the file's SHA-256 hash. If both uploads succeed with the same hash, there will be two catalog entries for the same content — a minor inefficiency (duplicated fragments) but not a correctness problem. A Last-Write-Wins CRDT timestamp resolves which entry is authoritative during gossip sync. In the current prototype we do not deduplicate storage, but it is a straightforward future optimisation.

Q34. What if the Bloom filter used in gossip keeps growing and becomes inaccurate? What If

Our Bloom filter is not persistent — it is rebuilt from scratch every gossip cycle from the current local DHT entries. So it never accumulates stale bits. The false-positive rate stays

stable because it only reflects what the node actually knows right now. A 1024-bit filter with 3 hash functions stays accurate up to about 100 entries; beyond that the false-positive rate climbs, which would mean a slight increase in unnecessary delta-sync requests — still not a correctness issue.

Q35. What if a node has very little battery left — does it still participate fully?

What If

This is not yet implemented as a dynamic behaviour in the current prototype. In the simulation (PeerSim), we modelled battery drain from gossip sends (0.5 units) and receives (0.3 units) to validate that fan-out=2 is a reasonable trade-off. In a production version, a low-battery node should reduce its gossip fan-out, refuse to accept new hosted blocks, and notify the Super-Peer so repairs can redirect traffic away from it before it goes offline.

Q36. What if someone replays an old election message to confuse the network?

What If

Every election message (ELECTION, ALIVE, COORDINATOR, ABDICATION) carries a timestamp and is signed with the sender's EC private key. On reception, the receiver checks that the timestamp is within ± 30 seconds of its own clock. A replayed message from 5 minutes ago will be rejected immediately as "outside the anti-replay window", even if the signature is valid.

Q37. What if a peer wants to leave the network gracefully?

What If

A Super-Peer broadcasts a signed ABDICATION message, then enters a 5-minute cooldown during which it cannot start or participate in a new election. This gives other nodes time to elect a replacement without the departing node interfering. A regular peer sends a LEAVE message signed with its identity key, which triggers the Super-Peer to remove it from the member list and proactively start repairing any blocks it was hosting.

Q38. What if the user's internet connection switches from Wi-Fi to 4G mid-transfer?

What If

The WebSocket connection to the relay will drop and reconnect. During reconnection (typically a few seconds), in-flight block transfers may fail. The upload use case handles each fragment independently — a failed fragment falls back to a second local peer or to an inter-cluster peer. The circuit breaker startup grace period (30 seconds) prevents a flood of false churn events when stale peer entries are evicted after reconnection.

Q39. What if you needed to scale to thousands of devices — would the current design hold? What If

Not without changes. The relay server is a bottleneck — a single Node.js process cannot handle thousands of persistent WebSocket connections without horizontal scaling and a shared session store (Redis, for example). The gossip protocol itself scales well ($O(\log N)$ convergence), but the single relay for signalling is the main weak point. The Super-Peer cluster model would need a second tier (Super-Peers of Super-Peers) to handle a global network, similar to Skype's historical architecture.

Q40. What if a user loses their phone — can they recover their files on a new device? What If

In the current prototype, no — because the private key is locked inside the TEE of the lost device and cannot be exported. This is actually the correct security property (the key cannot be stolen), but it means recovery requires a backup strategy. A production solution would involve a secure key backup mechanism, such as encrypting a key export with a user password and storing it on a trusted second device or a password-protected backup service.